

■ 1.8.8 Three-Dimensional Plotting

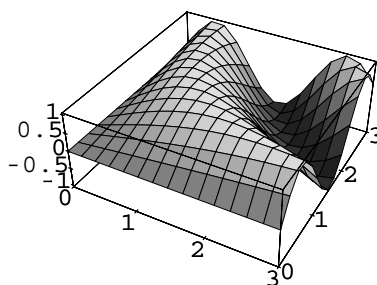
```
Plot3D[f, {x, xmin, xmax}, {y, ymin, ymax}]
```

make a three-dimensional plot of f as a function of the variables x and y

Basic 3D plotting function.

This makes a three-dimensional plot of the function $\sin(xy)$.

```
In[1]:= Plot3D[Sin[x y], {x, 0, 3}, {y, 0, 3}]
```



There are many options for three-dimensional plots in *Mathematica*.

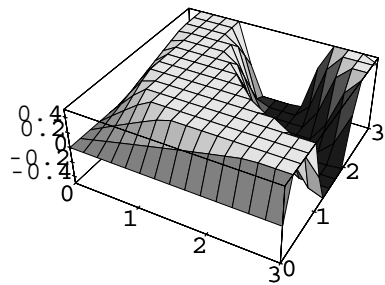
The first set of options are similar to those you use for standard two-dimensional plots.

option name	initial default value	
PlotRange	Automatic	the range of coordinates to include in the plot: you can specify <code>All</code> , <code>{zmin, zmax}</code> or <code>{{xmin, xmax}, {ymin, ymax}, {zmin, zmax}}</code>
PlotLabel	None	a label for the plot
Framed	False	whether to draw a frame around the plot
AspectRatio	1	the height-to-width ratio for the whole plot
PlotColor	True	whether to generate color graphics
PlotPoints	15	the number of points in each direction at which to sample the function; <code>{n_x, n_y}</code> specifies different numbers in the <i>x</i> and <i>y</i> directions

Some basic options for Plot3D. The first set of options can also be used in Show.

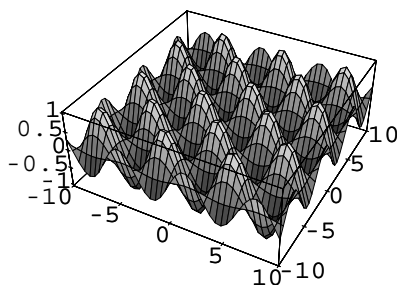
This redraws the plot on the previous line,
with options changed. With this setting for
PlotRange, only the part of the surface in
the range $-0.5 \leq z \leq 0.5$ is shown.

```
In[2]:= Show[%, PlotRange -> {-0.5, 0.5}]
```



When you make the original plot, you can choose to sample more points. You will need to do this to get good pictures of functions that wiggle a lot.

```
In[3]:= Plot3D[Sin[x] Sin[y], {x, -10, 10}, {y, -10, 10},
             PlotPoints -> 40]
```



You can think of the pictures you get from `Plot3D` as being like simulated photographs. There are several options for `Plot3D` and `Show` that parallel choices you would make in taking a real photograph.

option name	initial default value	
<code>ViewPoint</code>	<code>{1.3, -2.4, 2}</code>	the point from which to look at the surface
<code>Boxed</code>	<code>True</code>	whether to draw a box around the surface
<code>BoxRatios</code>	<code>{1, 1, 0.4}</code>	side length ratios for the box that encloses the surface

Options to `Plot3D` and `Show` for viewing surfaces.

The most important option is where you put the camera relative to the surface. You can see very different features of the surface by putting the camera in different places.

When *Mathematica* draws a three-dimensional object, it effectively encloses the object in a transparent cuboidal box. With the default setting `Boxed->True`, it draws the edges of this box explicitly. You can also squash or extend the box in different directions by setting the option `BoxRatios`.

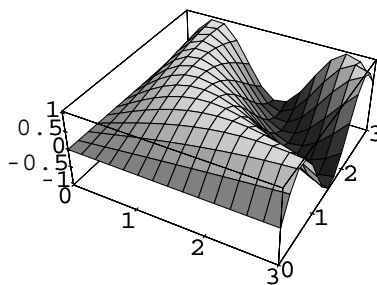
The `ViewPoint` option allows you to specify the coordinates $\{x, y, z\}$ of the camera relative to the center of the box. The coordinates are scaled so that the longest side of the box has length 1. If you use a cubical box, coordinates between $-\frac{1}{2}$ and $\frac{1}{2}$ will therefore be inside the box. The default setting for `ViewPoint` corresponds to a “generic position”, for which coincidental alignments between different parts of your object are unlikely.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

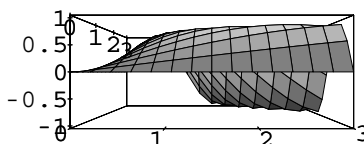
Here is a surface, viewed from the default “camera position” $\{1.3, -2.4, 2\}$.

```
In[4]:= g = Plot3D[Sin[x y], {x, 0, 3}, {y, 0, 3}]
```



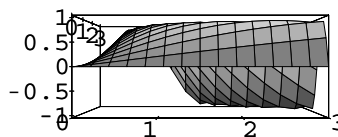
This redraws the picture, with the camera directly in front. Notice the perspective effect that makes the back of the box look much smaller than the front.

```
In[5]:= Show[%, ViewPoint -> {0, -2, 0}]
```



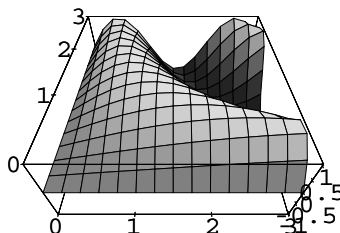
Moving the camera position further away reduces the perspective effect.

```
In[6]:= Show[%, ViewPoint -> {0, -4, 0}]
```



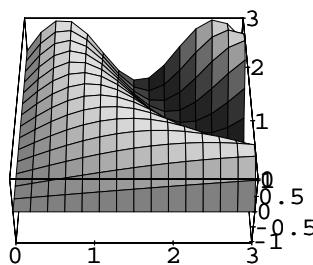
Here the camera points towards one of the sides of the box.

```
In[7]:= Show[%, ViewPoint -> {0, -1, 1}]
```



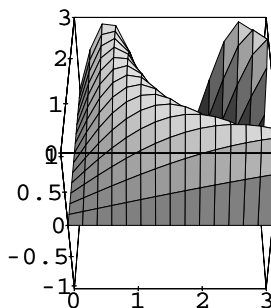
This moves the camera further away, reducing the perspective. The picture is automatically scaled so that the longest side of the box has the same apparent length.

```
In[8]:= Show[%, ViewPoint -> {0, -4, 4}]
```



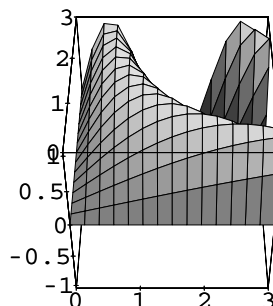
This shows the surface drawn in a cubical box.

```
In[9]:= Show[%, BoxRatios -> {1, 1, 1}]
```



Setting `Boxed->False` stops the box from being drawn.

```
In[9]:= Show[%, BoxRatios -> {1, 1, 1}]
```



<code>{0, -2, 0}</code>	directly in front
<code>{0, -2, 2}</code>	in front and up
<code>{0, -2, -2}</code>	in front and down
<code>{-2, -2, 0}</code>	left-hand corner
<code>{2, -2, 0}</code>	right-hand corner
<code>{0, 0, 2}</code>	directly above

Typical choices of the `ViewPoint` option.

The human visual system is not particularly good at understanding complicated mathematical surfaces. As a result, you need to generate pictures that contain as many clues as possible about the form of the surface.

Camera positions slightly above the surface are usually best. It is typically a good idea to keep the camera close enough to the surface that there is some perspective effect. Having a box explicitly drawn around the surface is helpful in recognizing the orientation of the surface.

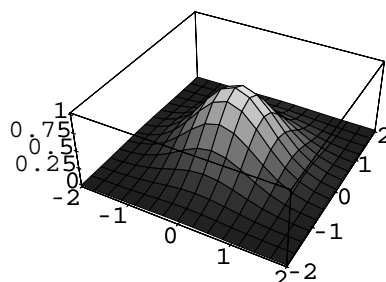
There are several options for how the actual surface should be shown in `Plot3D`.

<i>option name</i>	<i>default value</i>	
HiddenSurface	True	whether parts of the surface that are obscured should be hidden
Shading	True	whether the surface should be shaded, or left white
Mesh	True	whether an xy mesh should be drawn on the surface

Options for rendering surfaces in Plot3D and Show.

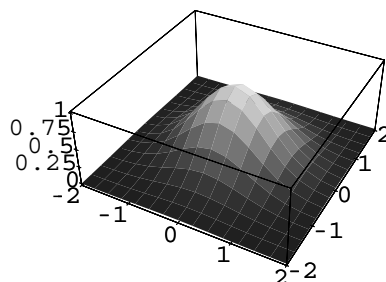
Here is a plot with the default settings for surface rendering options.

```
In[11]:= g = Plot3D[Exp[-(x^2+y^2)],
                  {x, -2, 2}, {y, -2, 2}]
```



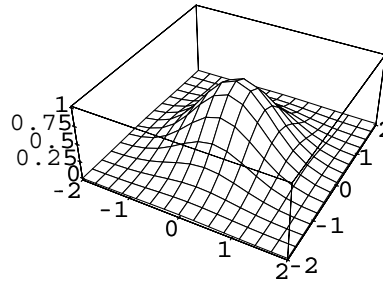
This shows the surface without the mesh drawn. It is usually much harder to see the form of the surface if the mesh is not there.

```
In[12]:= Show[g, Mesh -> False]
```



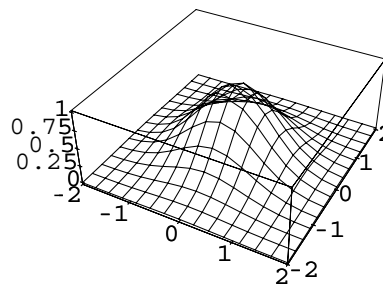
This shows the surface with no shading.
Some display devices may not be able to
show shading.

```
In[13]:= Show[g, Shading -> False]
```



Here is the picture without hidden surface
removal. The surface is now like a “wire
frame” that you can see through.

```
In[14]:= Show[%, HiddenSurface -> False]
```



The inclusion of shading and a mesh are usually great assets in understanding the form of a surface. On some vector graphics output devices, however, you may not be able to get shading. You should also realize that when shading is included, it may take a long time to render the surface on your output device.

A final element of realism that you can include in the three-dimensional pictures produced by *Mathematica* is simulated *illumination*. The darkness of shading at a particular point on the surface is usually determined just by its height. However, if you set `Lighting -> True`, *Mathematica* will shade the surface to simulate illumination. (There are nevertheless no shadows produced.)

<i>option name</i>	<i>default value</i>	
Lighting	False	whether to include simulated illumination
AmbientLight	GrayLevel[0.]	isotropic ambient light
LightSources	(see below)	directions and colors of point light sources, specified by $\{\{\{x_1, y_1, z_1\}, i_1\}, \{\{x_2, y_2, z_2\}, i_2\}, \dots\}$

Options for simulated illumination.

You can specify two components to the illumination of an object. First, you can set **AmbientLight** to specify diffuse isotropic lighting, which gives a uniform shading over the whole surface. Second, you can specify point light sources with particular positions and colors. The effect of all these light sources is added together to determine the shading of each point on the surface. The surface is always assumed to reflect all the light that reaches it.

If you have a color output device, *Mathematica* should be able to give you graphics in color. Simulated lighting is often a good way to introduce color into a picture. If you specify a color picture, the colors will simply be averaged to give a gray level when you use a black-and-white display device.

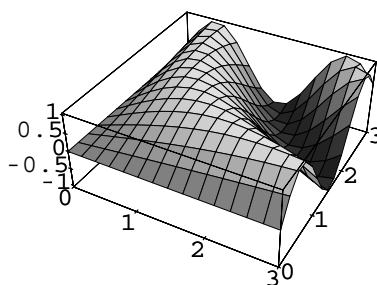
If you set **Lighting->True**, but do not explicitly set **AmbientLight** or **LightSources**, *Mathematica* will use its default arrangement of simulated lighting. This consists of three point light sources, respectively red, green and blue, with no ambient light. The point light sources are placed at 45° angles on the right-hand side of the object.

GrayLevel[gray]	gray level between 0 (black) and 1 (white)
RGBColor[r, g, b]	red, green, blue color intensities, each between 0 and 1

Mathematica gray level and color specifications.

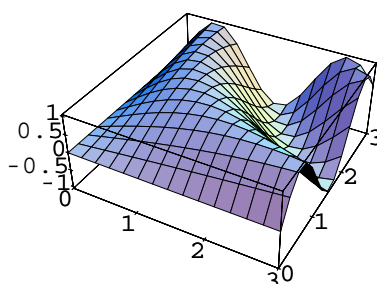
Unless you specify otherwise, `Plot3D` shades surfaces with gray levels determined by the height of each point on the surface.

```
In[15]:= Plot3D[Sin[x y], {x, 0, 3}, {y, 0, 3}]
```



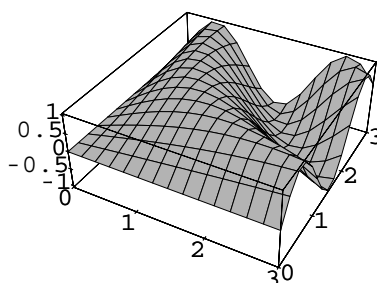
This shows the same surface, with shading determined by simulated illumination. The default illumination, used here, has three light sources of different colors on the upper right. A particular square on the surface is brightest if its orientation is normal to the direction of the light sources.

```
In[16]:= Show[%, Lighting -> True]
```



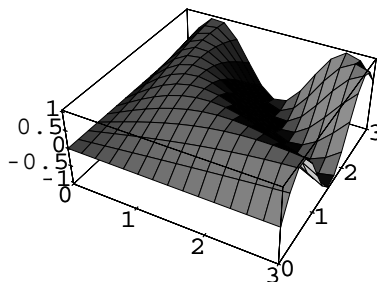
If you have no light sources, the surface is uniformly shaded with gray from the ambient light.

```
In[17]:= Show[%, AmbientLight -> GrayLevel[0.7],  
LightSources -> {}]
```



With no ambient light, parts of the surface away from the point light source are black.

```
In[18]:= Show[%, AmbientLight -> GrayLevel[0.],
             LightSources -> {{{-1, -1, 1}, GrayLevel[0.8]}}]
```



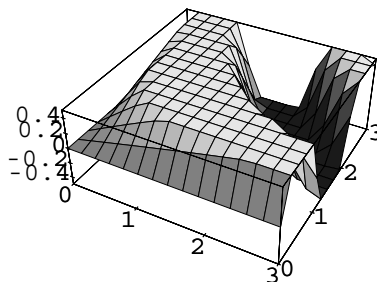
Another option that you can set in `Plot3D` concerns the way that “clipped” surfaces are shown. Clipping is simple in two-dimensional graphics: lines that extend outside the plot are just truncated. In three-dimensional graphics, clipping is more complicated. *Mathematica* always cuts off surfaces when they cross the rectangular box that it uses for three-dimensional pictures. The option `ClipFill` specifies what is shown in the places where the surface is clipped.

<code>None</code>	leave out clipped parts of the surface, so that you can see through
<code>Automatic</code>	show the clipped part of the surface with the same shading as an actual surface in the same position would have (default setting)
<code>GrayLevel[i]</code> or <code>RGBColor[r, g, b]</code>	show the clipped part of the surface with a particular gray level or color
<code>{bottom, top}</code>	give different specifications for parts that are clipped at the bottom and top

Settings for the `ClipFill` option.

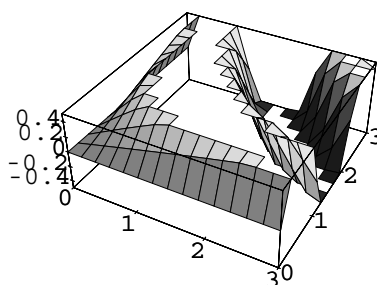
Here is a three-dimensional plot in which the top and bottom of the surface are clipped. With the default setting for `ClipFill`, the clipped parts are shown as they would be if they were part of the actual surface.

```
In[19]:= Plot3D[Sin[x y], {x, 0, 3}, {y, 0, 3},
  PlotRange -> {-0.5, .5}]
```



With `ClipFill->None`, parts of the surface which are clipped are left out, so that you can “see through” the surface there. *Mathematica* always leaves out parts of the surface that correspond to places where the value of the function you are plotting is not a real number.

```
In[20]:= Show[%, ClipFill -> None]
```



This makes the bottom clipped face white (gray level 1), and the top one black.

```
In[21]:= Show[%, ClipFill -> {GrayLevel[1], GrayLevel[0]}]
```

